# COMP2113 Programming Technologies

## Assignment 4

**Deadline: 3rd May at 23:59**

---

### General Instructions

Submit your assignment via VPL on Moodle. Ensure that your program can execute, and generate the required outputs in VPL. Work incompatible with the VPL may not be marked.

For C/C++ programs, ensure compilation with the gcc C11/C++11 standard on our standard environment. This is already enforced in VPL. If you are testing in your own environment, use the following commands to compile your programs:

For C++ programs:

```
g++ -pedantic-errors -std=c++11 -o [executable name] [yourprogram].cpp
```

For C programs:

```
gcc -pedantic-errors -std=c11 -o [executable name] [yourprogram].c
```

As a developer, ensure that your code works flawlessly in the intended environment, not just your own. While you may develop your work in your own environment, always test your program in our standard environment (i.e., the VPLs) before submission.

### Evaluation

For tasks requiring **user input**, utilize the **standard input**. Likewise, your program should **output/print** through the **standard output**. Strict adherence to the sample output format is required, or your answer may be marked incorrect.

Your code will be automatically graded for technical correctness. Essentially, we use test cases to evaluate your solution, failure to pass any of the test cases may result in zero marks. Partial credits are generally not given for incomplete solutions as it may be challenging to objectively assess incomplete program logic. However, your work may still be considered on a case-by-case basis during the rebuttal stage.

Additional test cases will be used during grading. Scoring full marks on VPL does not ensure full marks in the assignment. Sample test cases may or may not encompass all boundary cases. Designing proper test cases to verify your program's accuracy is part of the assessment.

### Academic Dishonesty

Your code will be cross-checked with other submissions and online sources for logical duplication. Note that providing your work to others, aiding others in copying, or copying from others will be considered plagiarism, and will be dealt with as per departmental policy. Please refer to the course information notes for more details.

**Use of generative AI tools, like ChatGPT, is not permitted for all assignments.**

### Getting Help

You are not alone! If you are stuck, post your query on the course forum. This assignment should be educational and rewarding, not frustrating. We are here to help, but we can only do so if you reach out.

**Please avoid spoilers on the discussion forum.** Do not post any code directly related to the assignments. You are, however, encouraged to discuss general concepts on the forums.

**Submission**

Deadlines are strictly enforced. Resubmission beyond the submission period will not be accepted.

Late Policy:

- If you submit within 2 days after the deadline, 30% deduction.
- If you submit within 3-5 days after the deadline, 50% deduction.
- After that, no mark.

---

## Problem 1: Trigram Analyzer

A trigram is a sequence of three adjacent elements from a string. For example the token "Trigram!" has 6 trigrams: `Tri`, `rig`, `igr`, `gra`, `ram`, `am!`.

Write a C++ program, `1.cpp`, that reads tokens from user input until an empty line is received. Count all trigrams in these tokens and output all unique tokens that contains the most frequently occurring trigram. While you may choose any method to solve this task, utilizing the Standard Template Library (STL) could simplify your solution.

**Input:**

- The program should read lines from user input until an empty line is received.
- All input lines, except the last one, consist of one or more tokens. Tokens are separated by one whitespace character, and may consist of any printable ASCII characters.
- The last input line will always be an empty line.

**Output:**

- The program should identify the most frequently occurring trigram and then output all **unique tokens** containing that trigram.
- Tokens must be listed in order of their first appearance.
- If there is no trigram in the input, the program outputs nothing.

**Requirements and Assumptions:**

- The program should be case sensitive. For instance, `Trigram!` and `trigram!` should be considered as two different tokens. Similarly, `Tri` and `tri` should be considered as two different trigrams.
- Trigrams may contain any printable ASCII characters, including digits and symbols. For instance, the token `x=1` has one trigram: `x=1`.
- Tokens with only one or two characters do not have a trigram.
- If multiple trigrams are the most frequently occurring, select the trigram that is first encountered as the most frequently occurring trigram.
- There is no length limit for the input. There could be as many tokens as that the VPL can handle.
- To avoid stability issues with VPLs and to adhere to memory limits, do not include `bits/stdc++.h`. Instead, include individual libraries as needed. Including `bits/stdc++.h` may cause your program to exceed memory limits and fail to compile.

**Sample Test Cases**

**1__1**

---

Input:

```
Python supports multiple programming
paradigms including procedural and
functional programming

```

Output: (trigram is `pro`)

```
programming procedural
```

**1_2**

| | |
|---|---|
| Input: | `row row row your boat gently down the stream merrily merrily merrily merrily life is but a dream` |
| Output:(trigram is `mer`) | `merrily` |

**1_3**

| | |
|---|---|
| Input: | `a b c d e f g` |
| Output: | *The program outputs nothing* |

## Problem 2: Run-length Encoding

"Run-length" refers to the number of times a symbol is repeated consecutively. For example, the string `abbaaa` is seen as `a` with a run-length of 1, then `b` with a run-length of 2, and finally another `a` with a run-length of 3.

Run-length encoding encodes a sequence of symbols based on their run-length. Consider the following encoding scheme:

- To encode a string, the string is first split into runs of consecutive identical symbols, where each run has a maximum length of 10. Runs are formed greedily from left to right, always taking the maximum possible length. For example, the string `abbbaaaaaaaaaaaaaa` is split into `a`, `bbb`, `aaaaaaaaaa`, `aaa`.
- For each run of symbols, encode it as the symbol followed by a single digit that equals the run-length minus 1. For instance, `a` will be encoded as `a0`, and `bbb` will be encoded as `b2`.

Using this encoding scheme, `abbbaaaaaaaaaaaaaa` will be encoded as `a0b2a9a2`.

Write a C program, `2.c`, that reads a string from user, encodes it using the above run-length encoding scheme, and outputs the encoded string.

**IMPORTANT: For this question, you must submit a C program. No mark would be given if you submit a C++ program.**

**Assumptions:**

- The input may contain letters, digits, or symbols. There will be no whitespaces in the input string.
- The input is always a valid string. The length of the input string is at least 1 character and at most 500 characters.

**Sample Test Cases**

**2_1**

| | |
|---|---|
| Input: | `abbbaaaaaaaaaaa` |
| Output: | `a0b2a9a0` |

**2_2**

| | |
|---|---|
| Input: | `**!!!!!!!!!` |
| Output: | `*1!7` |

**2_3**

| | |
|---|---|
| Input: | `ABCDEFGHIJKLMNOPQRSTUVWXYZ` |
| Output: | `AOBOCODOEOFOGOHOIOJOKOLOMONOOOPOQOROSOTOUOVOWOXOYOZO` |

---

## Problem 3: Prefix Notation Evaluation

Prefix notation presents operators before operands in a mathematical notation. For example, the expression `+ 1 2` in prefix notation is equivalent to 1 + 2 in conventional notation, and `+ * 2 3 * 5 6` is equivalent to (2 * 3) + (5 * 6).

A stack is a collection of elements that is accessed in a **last in, first out** order. It primarily supports two operations:

- **Push**: Append an element to the collection.
- **Pop**: Removes and returns the last element in the collection.

Expressions in prefix notation can be evaluated using a stack by reading the expression from **right to left**:

- Read one token from the expression, from right to left.
  - If the token is a number, push it to the stack.
  - If the token is an operator, pop two values from the stack. The first value popped is the left operand and the second value popped is the right operand. Apply the operation and push the result back to the stack.
- Repeat until no more token could be read. The result of the evaluation is at the top of the stack.

Write a C program, `3.c`, that evaluates an expression in prefix notation.

**IMPORTANT: For this question, you must submit a C program. No mark would be given if you submit a C++ program.**

### Input

- The program should read an expression in prefix notation from user input. A `=` sign is added to the end of expression to indicate the end of input.
- For this problem, all operands in the expression will be single digit. Therefore, space is eliminated from the input. For instance, (2 * 3) + (5 * 6) will be input as `+*23*56=`.

### Output

The program should:

- Implement a **dynamic array** and use it as a stack, with an initial capacity of 1.
- Read the entire expression from user input (up to and including the `=` sign). Then process the expression from **right to left** (excluding the `=` sign). Each character is considered as one token.
  - If the token is a single digit, push it to the stack. If the stack is full after the push, expand the stack by doubling its capacity.
  - If the token is an operator, which can be `+`, `-`, or `*`, apply the corresponding operation.
- For each token processed, the program should output the current capacity and the content of the stack. Refer to the example and the sample test cases below for the exact format.

**Example**

Input: `+*23*56=`

| State | Action | Stack | Capacity | Output |
|---|---|---|---|---|
| *initial* | Initialize a stack with a capacity of 1. | `[ ]` | 1 | *nil* |
| Read `6` | Push 6 to stack. Stack expanded as it is full. | `[6]` | 2 | `[2] 6` |
| Read `5` | Push 5 to stack. Stack expanded as it is full. | `[6, 5]` | 4 | `[4] 6 5` |
| Read `*` | Pop 5 and 6 from stack. Calculates 5 * 6 = 30, push 30 to stack. | `[30]` | 4 | `[4] 30` |
| Read `3` | Push 3 to stack. | `[30, 3]` | 4 | `[4] 30 3` |
| Read `2` | Push 2 to stack. | `[30, 3, 2]` | 4 | `[4] 30 3 2` |
| Read `*` | Pop 2 and 3 from stack. Calculates 2 * 3 = 6, push 6 to stack. | `[30, 6]` | 4 | `[4] 30 6` |
| Read `+` | Pop 6 and 30 from stack. Calculates 6 + 30 = 36, push 36 to stack. | `[36]` | 4 | `[4] 36` |
| *end* | All tokens processed. The result 36 is at the top of the stack. | | | |

**Notes on Memory Allocation:**

Include `stdlib.h` for memory allocation in C.

```c
#include <stdlib.h>
```

To implement a dynamic array in C, we can use `malloc()` to allocate the memory needed to a pointer. We use `sizeof()` to calculate the size of the memory. Note that since `malloc()` returns a generic `void *` type, it is necessary to cast it to a suitable type.

```c
int n = 1; // capacity of array
int * dynArr = (int *)malloc(n * sizeof(int)); // allocate memory
```

Note the values in the allocated array is not initialized. In some systems, the values are initially zero, but it is never a guarantee. You must initialize the values before using them.

To expand the dynamic array, we can use `realloc()` to reallocate the memory needed.

```c
dynArr = (int *)realloc(dynArr, n * sizeof(int)); // expand dynArr to store n integers
```

Finally, when the array is not needed anymore, we can free the allocated memory with `free()`.

```c
free(dynArr);
```

**Assumptions:**

- The input will only contain a single line.
- Tokens can only be one of the following: `0`, `1`, `2`, `3`, `4`, `5`, `6`, `7`, `8`, `9`, `+`, `-`, `*`, `=`.
- The last token in the input will always be `=`, all other tokens will not be `=`.
- Input expression will always be valid. When processing from right to left, the program will not encounter a `+`, `-`, or `*` when the stack has fewer than two elements. After all tokens are processed, the stack will have exactly one element.
- There is no length limit for the input. There could be as many tokens as that the VPL can handle.

**Sample Test Cases (See the example above for a detailed explanation)**

**3_1**

| | |
|---|---|
| Input: | `+*23*56=` |
| Output: | `[2] 6 [4] 6 5 [4] 30 [4] 30 3 [4] 30 3 2 [4] 30 6 [4] 36` |

**3_2**

| | |
|---|---|
| Input: | `7=` |
| Output: | `[2] 7` |

**3_3**

| | |
|---|---|
| Input: | `*+12+34=` |
| Output: | `[2] 4 [4] 4 3 [4] 7 [4] 7 2 [4] 7 2 1 [4] 7 3 [4] 21` |